



# 文件系统的遍历与文件查找

---

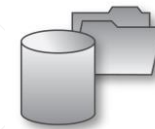
北京理工大学计算机学院  
金旭亮



# 遍历文件系统

---

# FileStore



在Linux中，每一个Mounted的文件系统，都是一个FileStore。  
在Windows中，每一个驱动器，都是一个FileStore。

以下示例代码，可列出所有的驱动器：

```
public class UseFileStore {  
    public static void main(String[] args) {  
        FileSystems.getDefault()  
            .getFileStores()  
            .forEach(System.out::println);  
    }  
}
```

```
Run: UseFileStore x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
OS (C:)  
DATA (D:)  
MyWebSite (F:)  
upload (G:)  
HD-working (H:)  
J_CCSA_X64F (M:)  
  
Process finished with exit code 0
```

# 以流的形式检索文件



```
//使用list方法列举指定文件夹下的文件
private static void listCurrentPathContent() throws IOException {
    Path curDir = Paths.get( first: "./src");
    try(var files : Stream<Path> = Files.list(curDir)){
        files.forEach(System.out::println);
    }
}
```

useFileTree.java



list方法传入Path作为参数，并返回一个包装了DirectoryStream的流对象。由于DirectoryStream接口继承自AutoCloseable，所以可用于try-with-resources结构中。



对非目录资源执行list方法将抛出NotDirectoryException。

# 读取文件夹中的文件

```
//使用DirectoryStream列举指定文件夹下的文件
private static void useDirectoryStream() throws IOException {
    Path curDir = Paths.get( first: "./src");
    try (var files : DirectoryStream<Path> = Files.newDirectoryStream(curDir)) {
        files.forEach(System.out::println);
    }
}
```

# 文件筛选



`Files.newDirectoryStream()` 方法有一个重载形式，可以用于筛选文件：

```
public static DirectoryStream<Path> newDirectoryStream(Path dir,  
    DirectoryStream.Filter<? super Path> filter)  
throws IOException
```



```
@FunctionalInterface  
public static interface Filter<T> {  
    boolean accept(T entry) throws IOException;  
}
```



这是一个函数式接口，  
所以，可以接收一个  
Lambda表达式。

# 使用DirectoryStream查找文件示例



```
private static void useDirectoryStreamToFind() throws IOException {  
    Path searchDir = Paths.get( first: "./src");  
    try (var files : DirectoryStream<Path> = Files.newDirectoryStream(searchDir,  
        file -> file.getFileName().toString().endsWith(".java"))) {  
        files.forEach(f -> {  
            System.out.println(f.getFileName());  
        });  
    }  
}
```

FindFiles.java

# 深度遍历文件夹

UseFileTree.java

```
//使用Files.walk方法深度遍历文件系统
private static void useWalk(Path fileDir) throws IOException {
    //使用Stream API, 可以得到最简单的深度遍历方式
    Files.walk(fileDir, FileVisitOption.FOLLOW_LINKS)
        .forEach(System.out::println);
}
```

walk方法的参数为起始Path以及FileVisitOption值的可变参数列表。从起始路径开始对文件系统执行深度优先遍历，返回一个由Path实例惰性填充的Stream。

FileVisitOption是Java 1.7引入的一个枚举类型，所包含的唯一枚举常量为FOLLOW\_LINKS。至少从理论上说，FOLLOW\_LINKS意味着文件树中可能存在循环，因此流将跟踪所访问的文件。一旦检测到循环，程序将抛出FileSystemLoopException。



# 定制文件遍历过程

如果需要比较灵活的处理，可以使用Files类所提供的walkFileTree方法遍历文件系统：

```
public static Path walkFileTree(Path start,  
                                FileVisitor<? super Path> visitor) throws IOException
```

- FileVisitor是一个接口，它定义了四个方法：
  - (1) preVisitDirectory：在访问文件夹前调用
  - (2) postVisitDirectory：在访问文件夹后调用
  - (3) visitFile：在访问文件时调用
  - (4) visitFileFailed：当指定文件不可访问时调用

应用实例：

```
private static void useFileVisitor(Path fileDir) throws IOException {  
    var fileVisitor = new MyFileVisitor();  
    Files.walkFileTree(fileDir, fileVisitor);  
}
```

UseFileTree.java

```
public class MyFileVisitor extends SimpleFileVisitor<Path> {  
    @Override  
    public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs)  
        throws IOException {  
        System.out.println("即将访问: "+dir);  
        return FileVisitResult.CONTINUE;  
    }  
    @Override  
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)  
        throws IOException {  
  
        if(attrs.isRegularFile()){  
            System.out.print("普通文件: ");  
        }  
        System.out.println(file);  
        return FileVisitResult.CONTINUE;  
    }  
}
```

MyFileVisitor.java

```
    @Override  
    public FileVisitResult visitFileFailed(Path file, IOException exc)  
        throws IOException {  
        System.out.println(exc.getMessage());  
        return FileVisitResult.CONTINUE;  
    }  
  
    @Override  
    public FileVisitResult postVisitDirectory(Path dir, IOException exc)  
        throws IOException {  
        System.out.println("访问完毕: "+dir);  
        return FileVisitResult.CONTINUE;  
    }  
}
```



# 查找文件

---

# 几种查找文件的方式

1

使用`Files.newDirectoryStream()`得到一个`DirectoryStream`实例，通过它实现文件查找功能。

2

使用`Files.find()`方法实现查找，可以通过`Lambda`表达式定义文件查找规则，比较简单。

3

使用`Files.walkFileTree()`方法实现查找，这种方法，需要自定义一个实现了`FileVisitor`接口的对象，重写其`visitFile()`方法，最复杂，但也最灵活。

# 文件系统的搜索

使用Files类定义的静态方法find查找文件，其定义如下：

```
public static Stream<Path> find(Path start,  
                                int maxDepth,  
                                BiPredicate<Path, BasicFileAttributes> matcher,  
                                FileVisitOption... options)  
    throws IOException
```

find方法从给定路径开始执行深度优先搜索（depthfirst search），直至达到maxDepth指定的目录级别。对于每条路径，find方法都会调用BiPredicate进行评估。

# find方法使用示例

```
private static void useFind() throws IOException {  
    //在当前文件夹下递归查找，列出所有的java文件  
    Path curDir = Paths.get("first: ".");  
    BiPredicate<Path, BasicFileAttributes> findWhat = (path, attributes) ->  
        !attributes.isDirectory() && path.toString().contains(".java");  
    try (Stream<Path> files = Files.find(curDir,  
        Integer.MAX_VALUE, findWhat)) {  
        files.forEach(System.out::println);  
    }  
}
```

# 使用PathMatcher查找文件



要在文件系统中查找文件，首先需要确定文件名的匹配规则。JDK 7中定义了一个PathMatcher接口，实现此接口的类可用于确定路径的匹配规范。



匹配规则有两类：一类是简单的诸如“\*.java”之类的通配符，另一类是正则表达式。



JDK中规定第一种匹配规则以“glob:”打头，第二种以“regex:”开头。

# 查找文件的示例

`FileSystems.getDefault().getPathMatcher()` 方法接收匹配字符串，返回一个可用的 `PathMatcher` 对象，程序之后就可以用它来进行文件名的匹配工作：

```
PathMatcher matcher = FileSystems.getDefault().getPathMatcher("glob:*.txt");
```



# 查找文件-1

```
public class FileFinder extends SimpleFileVisitor<Path> {
    private final PathMatcher matcher;
    public ArrayList<Path> foundPaths = new ArrayList<>();

    public FileFinder(String pattern) {
        matcher = FileSystems.getDefault()
            .getPathMatcher(syntaxAndPattern: "glob:" + pattern);
    }

    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)
        throws IOException {
        if (matcher.matches(file.getFileName())) {
            foundPaths.add(file);
        }
        return FileVisitResult.CONTINUE;
    }
}
```

FileFinder.java示例重写了JDK中所提供的SimpleFileVisitor<>类的visitFile方法，在内部使用PathMatcher查找所有Java源码文件。

# 查找文件-2

```
public class FindFiles {  
    public static void main(String[] args) throws IOException {  
        Path searchDir = Paths.get(first: ".");  
        FileFinder fileFinder = new FileFinder(pattern: "*.java");  
        Files.walkFileTree(searchDir, fileFinder);  
        fileFinder.foundPaths.forEach(System.out::println);  
    }  
}
```

# 动手动脑



1. 使用`Files.walkFileTree()`找出指定文件夹下所有大于指定大小（比如1M）的文件。
2. 使用`Files.walkFileTree()`找出指定文件夹下所有扩展名为`.txt`和`.java`的文件。
3. 使用`Files.walkFileTree()`找出指定文件夹下所有包容指定字符串的`txt`文件。